INSTITUTO FEDERAL
Brasília

Aula 5 – Push Button, tempo não bloqueante e Arrays

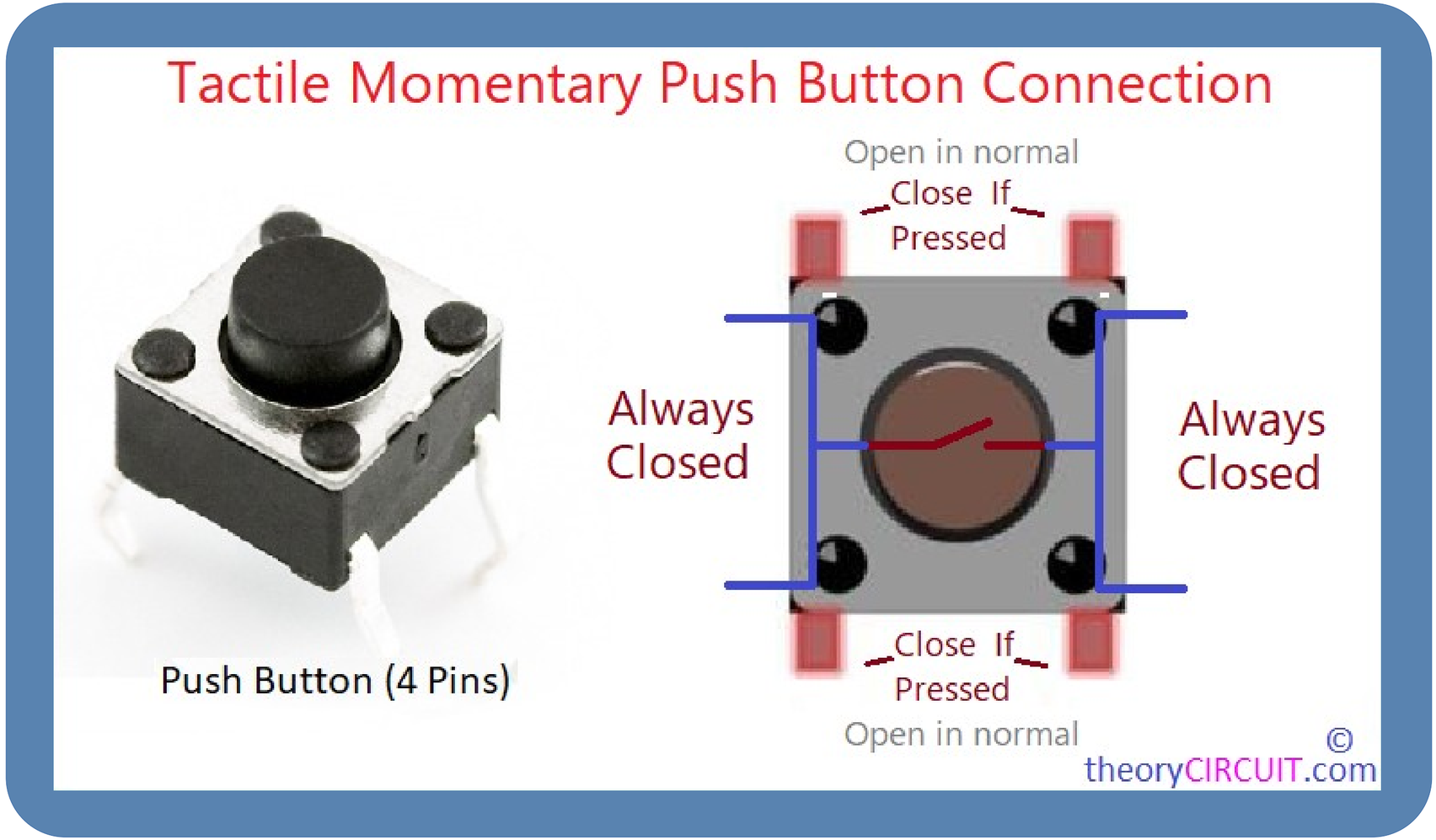
Marcos Santos
marcosvsantos@droidunb.com

INSTITUTO FEDERAL
Brasilía

Push Button

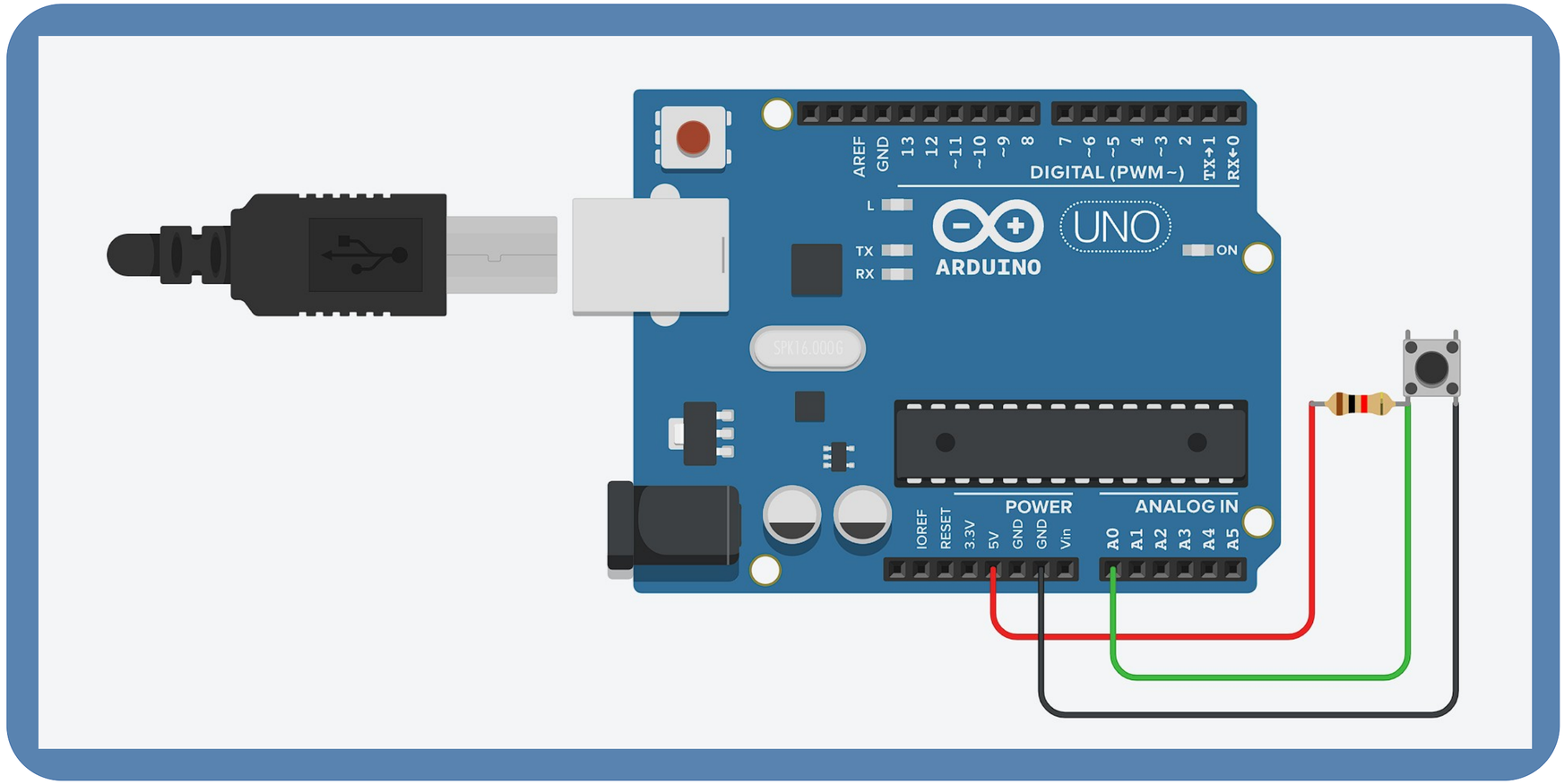
Conceito

Push Buttons são **chaves que só fecham o circuito enquanto são pressionados**. Possuem um par de dois conectores, que estão sempre ligados entre si. Quando pressionado, os quatro conectores passam a estar ligados.



Montagem

Nesse caso, sempre que pressionarmos o botão, o sinal será totalmente aterrado sem nenhum curto, ou seja, **A0 = 0**. **Sempre que soltarmos**, o sinal estará ligado ao VCC, ou seja, **A0 = 1** (5V).



Programação

Podemos realizar os comandos a depender do **valor em tempo real (sem delay)** que o Push Button retorna para o Arduino.

```
void setup()
{
  pinMode(A0,INPUT);
  Serial.begin(9600);
}

void loop()
{
  bool entrada = digitalRead(A0);
  Serial.print("botao status: ");
  if(entrada == true)
    Serial.println("desligado");
  else
    Serial.println("ligado");
}
```

Programação

Podemos fazer com que um **clique no Push Button altere o estado do programa entre ligado e desligado**.

```
void setup()
{
  pinMode(A0,INPUT);
  Serial.begin(9600);
}

void loop()
{
  int entrada = digitalRead(A0);
  if(entrada == 0){
    entrada = 1;
    delay(300);
    while(entrada == 1){
      entrada = digitalRead(A0);
      Serial.println("Led ligado");

    }
    delay(300);
  }
  else{
    Serial.println("Led desligado");
  }
}
```

Programação

Podemos **selecionar estados para uma variável a depender da quantidade de cliques** em nosso Push Button.

```
int estado = 1;

void setup()
{
  pinMode(A0,INPUT);
  Serial.begin(9600);
}

void loop()
{
  int entrada = digitalRead(A0);
  if(entrada == 0){
    estado++;
    if(estado > 4)
      estado = 1;
    delay(100);
  }
    Serial.println(estado);
}
```

INSTITUTO FEDERAL
Brasilía

Tempo não bloqueante

Noções de tempo não bloqueante

Um tempo não bloqueante é a funcionalidade de **pausar certas ações sem comprometer outras ações que estão rodando**.
Essa funcionalidade é muito diferente de **um simples delay(), que pausa todo o código** enquanto não se passar a quantidade de milissegundos como parâmetro.
Ou seja, **é literalmente um delay() que não trava tudo**, o que traz maior independência entre funcionalidades distintas que seu robô pode ter.
Geralmente **construímos essa funcionalidade a partir de condições tendo como valor base:** $t - t_0$.
t → o tempo total em que o Arduino está ligado capturado no momento de início
t_0→ o tempo total em que o Arduino está ligado instantaneamente

Uso de tempo não bloqueante

Podemos **contar o tempo em que o Arduino está ligado** através das funções **millis()** (que conta em milissegundos) e **micros()** (que conta em microssegundos).

```
unsigned int t;
unsigned int t0 = 0;
byte estado = true;

void setup()
{
  pinMode(13, OUTPUT);
}

void loop()
{
  t = millis();

  if((t - t0) == 1000){
    estado++;
    if(estado > 1)
      estado=0;
    t0 = t;
  }

  digitalWrite(13, estado);
}
```

INSTITUTO FEDERAL
Brasília

Arrays

Uma introdução técnica (pode ignorar por enquanto)

Arrays são blocos de memória. Esses blocos são repartidos em elementos com uma fração da memória total do bloco. A quantidade total de memória do bloco inteiro depende da quantidade de elementos especificada e do tipo desses elementos.

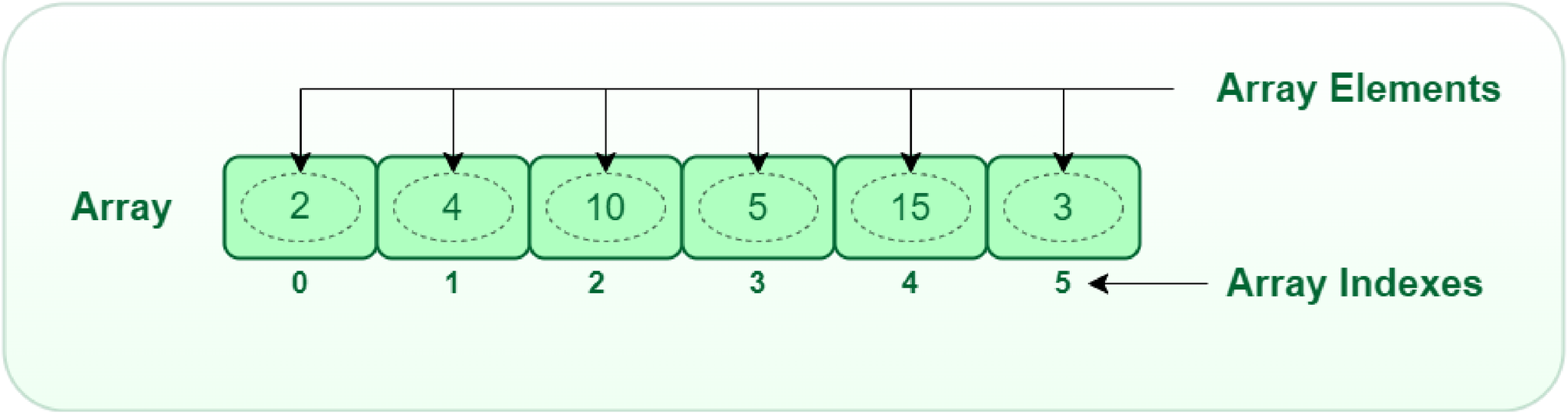
A medida que o bloco é repartido, cada elemento recebe seu próprio endereço. Sendo assim, o número do endereço do segundo é sucessor ao primeiro, e do terceiro sucessor ao segundo, e assim em série até o último elemento.

Na linguagem C, a variável que representa o array aponta para o endereço do primeiro elemento ***(diz ao compilador que o valor que queremos está dentro da memória naquele endereço)**. Se quisermos acessar outras posições, conseguimos especificar o índice delas, que se soma ao endereço do primeiro elemento, chegando ao elemento que queremos.

***lógica simplificada dos ponteiros**

Uma introdução conceitual

Resumindo, **arrays são blocos de memória repartidos em espaços para colocar elementos individuais**. Cada um desses elementos contém um **índex**, sua posição dentro da partição **(começando em 0)**, já que dividimos o bloco em série.



Programando arrays

Em C, devemos **criar o nosso array semelhantemente à uma variável**, porém utilizamos o [] para **dizer quantos elementos terá**. Você pode **atribuir valores dentro de {}** e entre vírgula para um array, mas isso **só é válido ao criar o array**. Se optar por colocar valores iniciais, não é necessário colocar a quantidade de elementos dentro de [].

```
1  void setup()
2  {
3    Serial.begin(9600);
4  }
5
6  void loop()
7  {
8    int a[3];
9    int b[3]={1,2,3};
10   int c[]={1,2,3};
11
12   for(int i = 0; i <= 2; i++){
13     Serial.println(b[i]);
14   }
15
16   Serial.println("");
17
18   b[2] = 10;
19   Serial.println(b[2]);
20
21   Serial.println("");
22
23   Serial.println(a[2]); //memória com "lixo", valor aleatório
24
25   Serial.println("");
26
27   delay(1000);
28
29 }
```

Programando arrays

Após criar o array, só conseguimos mudar elemento por elemento seu.

Para isso, escrevemos:

nome[index] = valor;

nome → nome do array

index → posição do elemento que quer alterar

valor → valor que se quer atribuir, respeitando o tipo do array

```
1  void setup()
2  {
3    Serial.begin(9600);
4  }
5
6  void loop()
7  {
8    int a[3];
9    int b[3]={1,2,3};
10   int c[]={1,2,3};
11
12   for(int i = 0; i <= 2; i++){
13     Serial.println(b[i]);
14   }
15
16   Serial.println("");
17
18   b[2] = 10;
19   Serial.println(b[2]);
20
21   Serial.println("");
22
23   Serial.println(a[2]); //memória com "lixo", valor aleatório
24
25   Serial.println("");
26
27   delay(1000);
28
29 }
```

Strings

Basicamente um array de caracteres (char). O último caractere desse tipo de array **é sempre '\0'**, o caractere nulo, utilizado para fechar a String.

Há **funções muito interessantes** envolvendo Strings, como a **strlen()**, que retorna a quantidade de caracteres de uma String (tirando o '\0').

```
void setup()
{
  Serial.begin(9600);
}

void loop()
{
  char parteUm[4]="Ola"; //isso é uma string
  char parteDois[]="Mundo!!"; //isso é uma string

  parteDois[5]=':'; //mudando um caractere isolado
  parteDois[6]=')'; //mudando um caractere isolado

  for(int i = 0; i<3; i++){
    Serial.print(parteUm[i]);
  }
  Serial.print(" ");

  for(int i = 0; i<strlen(parteDois); i++){
    Serial.print(parteDois[i]);
  }
  Serial.println("");
}
```

INSTITUTO FEDERAL
Brasilía

Parabéns pelo empenho!

Até a próxima aula!